

Gameplay Components

This page describes the high-level components used to connect MIDI input to Unity game logic.

Namespace: `jp.kshoji.unity.midi`

Location: `Assets/MIDI/Scripts/Gameplay/`

Overview

Component	Role
<code>MidiInputMap</code> / <code>MidiInputRouter</code>	Fire <code>UnityEvent</code> s from MIDI conditions defined in a <code>ScriptableObject</code>
<code>MidiNoteTracker</code>	Manage per-channel held-note state
<code>MidiChannelFilter</code>	Forward only specific channels downstream
<code>MidiDeviceFilter</code>	Forward only specific devices downstream
<code>SmfPlayer</code>	Synchronized SMF / Sequence playback (see SMF Tools)
<code>MidiRecorder</code>	Record live MIDI input as SMF (see SMF Tools)
<code>MidiCcSmoother</code>	Exponential moving-average smoothing of CC values
<code>MidiCcButton</code>	Button detection from a CC threshold
<code>MidiClockSync</code>	Sync to an external MIDI Clock; provide BPM / beat / bar
<code>MidiClockOutput</code>	Send MIDI Clock as master
<code>SmfPlayerClockAdapter</code>	Follow <code>SmfPlayer</code> tempo to an external Clock
<code>MidiChordDetector</code>	Detect chord-name / scale-membership changes and match against a target chord (quiz)

For Animator / Timeline / Visual Scripting integration, see [Unity Ecosystem Integration](#).

All of these use the event-handler mechanism of `MidiManager` (`IMidi*EventHandler`). Calling `InitializeMidi()` is the responsibility of the scene.

MidInputMap / MidInputRouter

Overview

Define mappings between MIDI messages and `UnityEvent`s in a `MidInputMap` (`ScriptableObject`), and `MidInputRouter` (`MonoBehaviour`) fires events when a condition matches. This lets you connect MIDI input to game events from the Inspector without writing code.

Setup

1. Create a `MidInputMap` from `Assets > Create > MIDI > Input Map`
2. Attach `MidInputRouter` to a `GameObject`
3. Assign the created Input Map to the `Map` field
4. Set conditions and `UnityEvents` in the Input Map's `Bindings`
5. Call `MidiManager.InitializeMidi()` in your scene bootstrap

Binding conditions

Field	Description
<code>messageType</code>	<code>MidiOutgoingMessageType</code> (<code>NoteOn</code> / <code>CC</code> / <code>PitchWheel</code> / <code>Start</code> / <code>SysEx</code> , etc.)
<code>group</code>	0–15, <code>-1</code> = all groups
<code>deviceIdFilter</code>	Empty = all devices
<code>channel</code>	0–15, <code>-1</code> = all channels (ignored for system triggers)
<code>noteNumber</code>	0–127, <code>-1</code> = any (<code>Note</code> / <code>Poly Aftertouch</code> / system data-byte filter)
<code>minVelocity</code> / <code>maxVelocity</code>	<code>NoteOn</code> velocity range
<code>controllerNumber</code>	<code>CC</code> number
<code>minValue</code> / <code>maxValue</code>	<code>CC</code> / <code>Aftertouch</code> value range
<code>ccTriggerMode</code>	<code>CC</code> trigger method (see table below)
<code>programNumber</code>	0–127, <code>-1</code> = any (for <code>PC</code>)

Message types and events

Category	messageType example	Fired event
Performance	NoteOn/Off, CC, PitchWheel, Aftertouch	<code>onTriggered</code> + <code>onTriggeredWithValue</code>
Performance	ProgramChange, SongSelect, ...	<code>onTriggered</code> + <code>onTriggeredWithValue</code> (<code>number</code>)
System trigger	TimingClock, Start, Stop, Reset, ...	<code>onTriggered</code> only
Payload	SystemExclusive, SystemCommonMessage, Midi2RawUmp	<code>onMessage</code> only

`MidiInputRouter` itself also has `raiseMessageEvents` / `onMessage` , letting you receive all incoming messages as a DTO (`MidiInputMessageEventArgs`). Use this for generic handling of SysEx / Raw UMP.

The legacy `MidiBindingMessageType` asset is migrated automatically via `FormerlySerializedAs` (PitchBend → PitchWheel).

CC trigger modes

Mode	Behavior
<code>OnChange</code>	Every time the value changes within range
<code>OnEnterRange</code>	The moment the value becomes <code>minValue</code> or higher
<code>OnExitRange</code>	The moment the value becomes <code>maxValue</code> or lower
<code>OnThreshold</code>	The moment the value becomes <code>minValue</code> or higher (for buttons)

Usage example

```
// Configure MidiInputMap in the Inspector and wire game logic to the UnityEvent
// Example: NoteOn ch0 note60 → turn on a light
```

Configuration example for a CC button (fires once at 64 or higher):

- `messageType` : `ControlChange`
- `channel` : 0
- `controllerNumber` : 64
- `minValue` : 64

- `ccTriggerMode : OnThreshold`

Notes

- Multiple bindings are evaluated with OR (fires if any one matches)
- If both `onTriggered` (no argument) and `onTriggeredWithValue` (int argument) are set, both fire
- When used via a filter, set `registerWithMidiManager = false` ([Combining with filters](#))

MidiNoteTracker

Overview

Tracks the note numbers currently held (Note On received, Note Off not yet received) per MIDI channel. It can serve as a foundation for chord detection, simultaneous-press counting, key-release detection, and so on.

Settings

Field	Description
<code>targetChannels</code>	Channels to track (empty = all channels)
<code>deviceIdFilter</code>	Empty = all devices
<code>registerWithMidiManager</code>	Whether to register directly with <code>MidiManager</code> (default true)
<code>onNoteStateChanged</code>	UnityEvent on note-state change

API

Method	Description
<code>IsNoteOn(channel, note)</code>	Whether the given note is held
<code>GetActiveNotes(channel)</code>	List of held notes
<code>GetActiveNoteCount(channel)</code>	Number of held notes
<code>GetTotalActiveNoteCount()</code>	Total across all channels
<code>HasAnyNoteOn()</code>	Whether any note is held

On receiving CC 120 (All Sound Off) and CC 123 (All Notes Off), all notes on that channel are treated as Off.

Usage example

```
using jp.kshoji.unity.midi;

public sealed class ChordDetector : MonoBehaviour
{
    public MidiNoteTracker tracker;

    void Update()
    {
        if (tracker.GetActiveNoteCount(0) >= 3)
            Debug.Log("3 notes or more pressed on channel 0");
    }
}
```

MidiChannelFilter / MidiDeviceFilter

Overview

Filter components that pass through only specific MIDI channels or devices and forward events to downstream GameObjects. In addition to all MIDI 1.0 messages, they also forward Raw UMP (`IMidi2RawUmpEventHandler`). Use them to split routes between UI and gameplay, for example.

Settings

MidiChannelFilter

Field	Description
<code>allowedChannels</code>	Allowed channels (empty = pass all)
<code>blockedChannels</code>	Blocked channels (takes priority over allowed)
<code>forwardTargets</code>	Array of forward-destination GameObjects

MidiDeviceFilter

Field	Description
<code>allowedDeviceIds</code>	Allowed device IDs (empty = pass all)
<code>blockedDeviceIds</code>	Blocked device IDs
<code>allowVirtualDevices</code>	Allow virtual devices with the <code>virtual:</code> prefix
<code>forwardTargets</code>	Array of forward-destination GameObjects

Combining with filters

Example wiring to pass events to downstream components through filters:

```
[MidiManager]
  ↓
[MidiDeviceFilter] ← registerWithMidiManager = true (chain entry point)
  ↓ forwardTargets
[MidiChannelFilter] ← registerWithMidiManager = false
  ↓ forwardTargets
[MidiInputRouter]  ← registerWithMidiManager = false
```

Important: When using a filter chain, set `registerWithMidiManager = false` on any `MidiChannelFilter` other than the entry point, and also set `registerWithMidiManager = false` on downstream `MidiInputRouter` and `MidiNoteTracker`. Registering twice causes events to be processed twice.

See `Assets/MIDI/Samples/Gameplay/Scenes/MidiGameplaySampleScene.unity` for a working example.

When using them directly without a filter, leaving `registerWithMidiManager = true` (the default) on the Router / Tracker is fine.

MidiClockSync / Chord & scale detection

MidiClockSync

Syncs to a MIDI Clock from an external sequencer, etc. BPM, beat, and bar events can be used from game logic or `SmfPlayerClockAdapter`.

Property / API	Description
<code>pulsesPerQuarterNote</code>	Clock pulses per quarter note (MIDI standard 24)
<code>beatsPerBar</code>	Beats per bar (default 4)
<code>deviceIdFilter</code>	Empty = all devices
<code>EstimatedBpm</code> / <code>IsBpmStable</code>	Estimated BPM value and stability flag
<code>onBeat</code> / <code>onBar</code>	Beat / bar boundary
<code>onStarted</code> / <code>onStopped</code>	Start / Stop received

```
var clock = gameObject.AddComponent<MidiClockSync>();
clock.onBeat.AddListener(() => Debug.Log("Beat"));
clock.onBar.AddListener(() => Debug.Log("Bar"));
```

`MidiClockOutput` can send a 120 BPM Clock. `SmfPlayerClockAdapter` links the tempo or playback position of `SmfPlayer` to an external Clock in `Follow` / `Step` / `Free` mode.

Chord & scale detection

Combine with `MidiNoteTracker` to estimate chord names and detect out-of-scale notes.

```
using jp.kshoji.unity.midi.util;

var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 0, MidiScaleType.Major);
```

`MidiChordDetector` notifies chord changes via `onChordChanged` and detects out-of-scale input via `onScaleViolation`. If you set `targetChord` and call `EvaluateNow()`, the match against the target chord (quiz) is notified via `onQuizCorrect` / `onQuizIncorrect`. `onAllNotesInScale` fires when all held notes are within the scale.

MidiCcSmoother / MidiCcButton

Provide smoothing and threshold toggling of CC values. They can also be used from Animator integration, etc.

MidiCcProcessorBase

Abstract base for CC processor components. Channel / device matching helpers and EMA / time-based smooth helpers (EMA delegates to `MidiControlSmoothing`).

Location: `Assets/MIDI/Scripts/Gameplay/MidiCcProcessorBase.cs`

API	Description
<code>MatchesChannel</code> / <code>MatchesDevice</code>	Filter helpers (-1 / empty = match all)
<code>ApplyEma(...)</code>	Delegates to <code>MidiControlSmoothing.ApplyEma</code>
<code>SmoothTowards(current, target, smoothingTime, deltaTime)</code>	Frame-based lerp toward a normalized target

MidiCcSmoother

Item	Description
<code>configs[]</code>	Controller number, channel, and <code>smoothFactor</code> (0.01–1.0; smaller is smoother)
<code>onSmoothedValue[]</code>	UnityEvent of the normalized value (0–1)
<code>GetSmoothedNormalizedValue(index)</code>	Get the latest normalized value

MidiCcButton

Item	Description
<code>onThreshold</code> / <code>offThreshold</code>	Threshold with hysteresis (default 64 / 63)
<code>onPressed</code> / <code>onReleased</code>	Press / release events
<code>IsPressed</code>	Current on-state

Source files

File	Role
<code>MidiInputMap.cs</code>	ScriptableObject definition
<code>MidiInputBinding.cs</code>	Binding definition
<code>MidiInputCondition.cs</code>	Condition-evaluation logic
<code>MidiInputBindingMigration.cs</code>	Legacy enum migration
<code>MidiInputMessageEventArgs.cs</code>	<code>onMessage</code> DTO
<code>MidiInputRouter.cs</code>	Event router
<code>MidiNoteTracker.cs</code>	Note-state management
<code>MidiNoteStateChangedEvent.cs</code>	State-change event args
<code>MidiFilterBase.cs</code>	Filter base class
<code>MidiChannelFilter.cs</code>	Channel filter
<code>MidiDeviceFilter.cs</code>	Device filter
<code>MidiEventForwarder.cs</code>	Event-forwarding helper
<code>SmfPlayer.cs</code> / <code>SmfPlayerState.cs</code>	SMF playback
<code>MidiRecorder.cs</code>	SMF recording
<code>MidiClockSync.cs</code> / <code>MidiClockState.cs</code> / <code>MidiClockOutput.cs</code>	MIDI Clock sync
<code>SmfPlayerClockAdapter.cs</code>	Tempo linkage between SMF and external Clock
<code>MidiChordDetector.cs</code>	Chord & scale detection (realtime detection + quiz)
<code>MidiCcProcessorBase.cs</code>	Shared base for CC processors
<code>MidiCcSmoother.cs</code> / <code>MidiCcButton.cs</code>	CC smoothing / threshold toggle
<code>Gameplay/Theory/ChordRecognition.cs</code> / <code>Gameplay/Theory/ScaleUtility.cs</code>	Chord-name estimation / scale utilities
<code>MidiManagerReceiverBridge.cs</code>	Sequencer → MidiManager bridge

Related docs

- [MIDI 1.0 Runtime \(MidiManager\)](#)
- [Utilities \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [Editor Tools \(MIDI Monitor\)](#)
- [SMF Tools \(SmfPlayer / MidiRecorder\)](#)
- [Getting Started Guide](#)